

Heaven's light is our guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Sessional Lab Report

Course Name : Data Mining Sessional

Course Code : CSE 4120

Date of Submission : 12 July, 2026

Submitted by :	Submitted to:
Name : Md. Asad-Al-Adil	Sadia Zaman Mishu
Roll no : 2103064	Assistant Professor
Section : B	Department of Computer Science and
Series : 21	Engineering, RUET

Content

Experiment No.	Experiment Name	Page No.
01	Dataset Selection and Data Preprocessing	3-10
02	Exploratory Data Analysis (EDA)	10-15
03	Mining Frequent Pattern and Association Rule	16-21
04	Application of Classification Techniques	21-33

Experiment-01: Dataset Selection and Data Preprocessing:

Introduction:

In the real world, data is rarely neat, tidy, or ready to be analyzed. It is often collected from various sources and comes filled with missing values, human typos, extreme outliers, and inconsistent formats. Data preprocessing is the critical first step in any data science or machine learning pipeline. It is the process of cleaning, transforming, and organizing raw data into a structured format that algorithms can actually understand and learn from. Some Significance of Data preprocessing:

- Prevents Model Crashes
- Improves Accuracy and Performance
- Feature Selection
- Reduces Complexity

Dataset Link- <https://archive.ics.uci.edu/dataset/20/census+income>

Dataset Description:



Census Income

Donated on 4/30/1996

Predict whether income exceeds \$50K/yr based on census data. Also known as Adult dataset.

Dataset Characteristics Multivariate	Subject Area Social Science	Associated Tasks Classification
Feature Type Categorical, Integer	# Instances 48842	# Features 14

Variables Table							^
Variable Name	Role	Type	Demographic	Description	Units	Missing Values	
age	Feature	Integer	Age	N/A		no	
workclass	Feature	Categorical	Income	Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked.		yes	
fnlwgt	Feature	Integer				no	
education	Feature	Categorical	Education Level	Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th-8th, 12th, Masters, 1st-4th, 10th, Doctorate, 5th-6th, Preschool.		no	
education-num	Feature	Integer	Education Level			no	
marital-status	Feature	Categorical	Other	Married-civ-spouse, Divorced, Never-married, Separated, Widowed, Married-spouse-absent, Married-AF-spouse.		no	

occupation	Feature	Categorical	Other	Tech-support, Craft-repair, Other-service, Sales, Exec-managerial, Prof-specialty, Handlers-cleaners, Machine-op-inspct, Adm-clerical, Farming-fishing, Transport-moving, Priv-house-serv, Protective-serv, Armed-Forces.	yes
relationship	Feature	Categorical	Other	Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried.	no
race	Feature	Categorical	Race	White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black.	no
sex	Feature	Binary	Sex	Female, Male.	no
capital-gain	Feature	Integer			no
capital-loss	Feature	Integer			no
hours-per-week	Feature	Integer			no

native-country	Feature	Categorical	Other	United-States, Cambodia, England, Puerto-Rico, Canada, Germany, Outlying-US(Guam-USVI-etc), India, Japan, Greece, South, China, Cuba, Iran, Honduras, Philippines, Italy, Poland, Jamaica, Vietnam, Mexico, Portugal, Ireland, France, Dominican-Republic, Laos, Ecuador, Taiwan, Haiti, Columbia, Hungary, Guatemala, Nicaragua, Scotland, Thailand, Yugoslavia, El-Salvador, Trinidad&Tobago, Peru, Hong, Holand-Netherlands.	yes
income	Target	Binary	Income	>50K, <=50K.	no

Import Necessary Library:

```
# import libraries:
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
```

Dataset Fetching from UCI:

```
# Dataset fetching
from ucimlrepo import fetch_ucirepo
# fetch dataset
census_income=fetch_ucirepo(id=20)
X=census_income.data.features
y=census_income.data.targets
X.head()
```

Dataset Display:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race	sex	capital-gain	capital-loss	hours-per-week	native-country
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba

Dataset Information and Shape:

```
X.info()  
X.shape
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 48842 entries, 0 to 48841  
Data columns (total 14 columns):  
#   Column                Non-Null Count  Dtype    
---  -  
0   age                    48842 non-null  int64    
1   workclass              47879 non-null  object   
2   fnlwgt                 48842 non-null  int64    
3   education              48842 non-null  object   
4   education-num          48842 non-null  int64    
5   marital-status         48842 non-null  object   
6   occupation             47876 non-null  object   
7   relationship           48842 non-null  object   
8   race                   48842 non-null  object   
9   sex                    48842 non-null  object   
10  capital-gain           48842 non-null  int64    
11  capital-loss           48842 non-null  int64    
12  hours-per-week         48842 non-null  int64    
13  native-country         48568 non-null  object   
dtypes: int64(6), object(8)  
memory usage: 5.2+ MB  
  
(48842, 14)
```

Displaying Total Number of Null value of every Feature:

```
X.isnull().sum()
```

dtype: int64

	0
age	0
workclass	963
fnlwgt	0
education	0
education-num	0
marital-status	0
occupation	966
relationship	0
race	0
sex	0
capital-gain	0
capital-loss	0
hours-per-week	0
native-country	274

Display the Unique items of categorical Features:

```
# Unique values in categorical column
categorical_columns=X.select_dtypes(include=['object']).columns
for i in categorical_columns:
    print(f"Unique values in '{i}': {X[i].unique()}\n")
```

Unique values in 'workclass': ['State-gov' 'Self-emp-not-inc' 'Private' 'Federal-gov' 'Local-gov' '?'
'Self-emp-inc' 'Without-pay' 'Never-worked' nan]

Unique values in 'education': ['Bachelors' 'HS-grad' '11th' 'Masters' '9th' 'Some-college' 'Assoc-acdm'
'Assoc-voc' '7th-8th' 'Doctorate' 'Prof-school' '5th-6th' '10th'
'1st-4th' 'Preschool' '12th']

Unique values in 'marital-status': ['Never-married' 'Married-civ-spouse' 'Divorced' 'Married-spouse-absent'
'Separated' 'Married-AF-spouse' 'Widowed']

Unique values in 'occupation': ['Adm-clerical' 'Exec-managerial' 'Handlers-cleaners' 'Prof-specialty'
'Other-service' 'Sales' 'Craft-repair' 'Transport-moving'
'Farming-fishing' 'Machine-op-inspct' 'Tech-support' '?'
'Protective-serv' 'Armed-Forces' 'Priv-house-serv' nan]

Unique values in 'relationship': ['Not-in-family' 'Husband' 'Wife' 'Own-child' 'Unmarried' 'Other-relative']

Unique values in 'race': ['White' 'Black' 'Asian-Pac-Islander' 'Amer-Indian-Eskimo' 'Other']

Unique values in 'sex': ['Male' 'Female']

Unique values in 'native-country': ['United-States' 'Cuba' 'Jamaica' 'India' '?' 'Mexico' 'South'
'Puerto-Rico' 'Honduras' 'England' 'Canada' 'Germany' 'Iran'
'Philippines' 'Italy' 'Poland' 'Columbia' 'Cambodia' 'Thailand' 'Ecuador'
'Laos' 'Taiwan' 'Haiti' 'Portugal' 'Dominican-Republic' 'El-Salvador'
'France' 'Guatemala' 'China' 'Japan' 'Yugoslavia' 'Peru'
'Outlying-US(Guam-USVI-etc)' 'Scotland' 'Trinidad&Tobago' 'Greece'
'Nicaragua' 'Vietnam' 'Hong' 'Ireland' 'Hungary' 'Holand-Netherlands' nan]

Handling the Missing or Null values:

```
# Preprocessing: Handling missing values and '?' value
null_columns=[]
for i in X.columns:
    if X[i].isnull().sum()>0:
        null_columns.append(i)
print(f"Columns with missing values: {null_columns}")

null_categories_col=[]
for i in null_columns:
    if X[i].dtype=='str':
        null_categories_col.append(i)
print(f"Categorical columns with missing values: {null_categories_col}")

for i in null_categories_col:
    X.fillna({i:X[i].mode()[0]}, inplace=True)

X['workclass']=X['workclass'].replace({'?':'Unknown'})
X['occupation']=X['occupation'].replace({'?':'Unknown'})

X.isnull().sum()
```

Columns with missing values: ['workclass', 'occupation', 'native-country']

Categorical columns with missing values: ['workclass', 'occupation', 'native-country']

	0
age	0
workclass	0
fnlwgt	0
education	0
education-num	0
marital-status	0
occupation	0
relationship	0
race	0
sex	0
capital-gain	0
capital-loss	0
hours-per-week	0
native-country	0

dtype: int64

Explanation:

- Find out the Null columns first.
- From Null Columns determine the categorical null columns

- Fill the categorical null value with mode value of each category.
- Replace ‘?’ value with Unknown.
- From the output there is no null value in numerical columns.

Feature value Substitution and Scaling:

```
# Preprocessing: Substitute Country name with continent name
def get_continent(country):
    continent_mapping = {
        'United-States': 'North America', 'Cuba': 'North America', 'Jamaica': 'North America',
        'India': 'Asia', 'Mexico': 'North America', 'South': 'Asia', 'Puerto-Rico': 'North America',
        'Honduras': 'North America', 'England': 'Europe', 'Canada': 'North America', 'Germany': 'Europe',
        'Iran': 'Asia', 'Philippines': 'Asia', 'Italy': 'Europe', 'Poland': 'Europe', 'Columbia': 'South America',
        'Cambodia': 'Asia', 'Thailand': 'Asia', 'Ecuador': 'South America', 'Laos': 'Asia', 'Taiwan': 'Asia',
        'Haiti': 'North America', 'Portugal': 'Europe', 'Dominican-Republic': 'North America',
        'El-Salvador': 'North America', 'France': 'Europe', 'Guatemala': 'North America',
        'China': 'Asia', 'Japan': 'Asia', 'Yugoslavia': 'Europe',
        'Peru': 'South America', 'Outlying-US(Guam-USVI-etc)': 'North America',
        'Scotland': 'Europe', 'Trinidad&Tobago': 'North America',
        'Greece': 'Europe', 'Nicaragua': 'North America', 'Vietnam': 'Asia',
        'Hong': 'Asia', 'Ireland': 'Europe', 'Hungary': 'Europe', 'Holand-Netherlands': 'Europe'
    }
    try:
        return continent_mapping[country]
    except:
        return 'Unknown'
X['native-country'] = X['native-country'].apply(get_continent)
df=X.copy()
# Preprocessing: Scaling the ages:
num_col=['age','fnlwgt','capital-gain','hours-per-week','capital-loss','education-num']
min_max_scaler=MinMaxScaler()
X[num_col]=min_max_scaler.fit_transform(X[num_col])
X.head()
```

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race	sex	capital-gain	capital-loss	hours-per-week	native-country
0	0.301370	State-gov	0.044131	Bachelors	0.800000	Never-married	Adm-clerical	Not-in-family	White	Male	0.02174	0.0	0.397959	North America
1	0.452055	Self-emp-not-inc	0.048052	Bachelors	0.800000	Married-civ-spouse	Exec-managerial	Husband	White	Male	0.00000	0.0	0.122449	North America
2	0.287671	Private	0.137581	HS-grad	0.533333	Divorced	Handlers-cleaners	Not-in-family	White	Male	0.00000	0.0	0.397959	North America
3	0.493151	Private	0.150486	11th	0.400000	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0.00000	0.0	0.397959	North America
4	0.150685	Private	0.220635	Bachelors	0.800000	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0.00000	0.0	0.397959	North America

Explanation:

- Replace the the ‘native-country’ values with the corresponding continent name of a country.
- It will helpful for one hot encoding and less number of feature will be created.
- After that Scaling technique is introduced named Min-Max Scaling. It makes the numerical column’s value in between [0,1]. This helps to remove outlier and make the region bounded to properly fit in the model.

$$\text{Min-Max} = \frac{\text{value} - \text{value}_{\min}}{\text{value}_{\max} - \text{value}_{\min}}$$

One hot encoding:

```
# Preprocessing: One Hot Encoding:
X_encoded=pd.get_dummies(X, columns=categorical_columns,dtype=int,drop_first=True)
print(X_encoded.head())
X_encoded.shape
```

	age	fnlwgt	education-num	capital-gain	capital-loss	\
0	0.301370	0.044131	0.800000	0.02174	0.0	
1	0.452055	0.048052	0.800000	0.00000	0.0	
2	0.287671	0.137581	0.533333	0.00000	0.0	
3	0.493151	0.150486	0.400000	0.00000	0.0	
4	0.150685	0.220635	0.800000	0.00000	0.0	

	hours-per-week	workclass_Local-gov	workclass_Never-worked	\
0	0.397959	0	0	
1	0.122449	0	0	
2	0.397959	0	0	
3	0.397959	0	0	
4	0.397959	0	0	

	workclass_Private	workclass_Self-emp-inc	...	relationship_Wife	\
0	0	0	...	0	
1	0	0	...	0	
2	1	0	...	0	
3	1	0	...	0	
4	1	0	...	1	

	race_Asian-Pac-Islander	race_Black	race_Other	race_White	sex_Male	\
0	0	0	0	1	1	
1	0	0	0	1	1	
2	0	0	0	1	1	
...						
3	0	0	0	0		
4	0	0	0	0		

[5 rows x 63 columns]

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)..

(48842, 63)

Explanation:

- One hot encoding is a preprocessing technique that convert the string type value to numerical as machine learning models are only accept numerical value of every features.
- This process create separate column for every unique items and fill them with 0,1 values.
- If any tuple contains any of them, it will put 1 for that tuple and other items will be 0.

Target variable Encoding:

```
# Preprocessing: Target variable encoding
print(y['income'].unique())

y['income'] = y['income'].map({'>50K': 1, '<=50K': 0, '<=50K.':0, '>50K.':1})

y.value_counts()
```

```
[ '<=50K' '>50K' '<=50K.' '>50K.' ]
```

	count
income	
0	37155
1	11687

dtype: int64

Discussion:

In this experiment I have used some preprocessing technique on my selected dataset to make it for model train easily like **Null Value Filling**, **Value replace with using function**, **Min-Max Scaling**, **One Hot Encoding**, **Target value mapping with Binary value (0,1)** as it is a binary classification problem. Now this Dataset Ready to fit in models for predicting different kinds of test cases and find out the test parameters value.

Experiment-2: Exploratory Data Analysis (EDA):

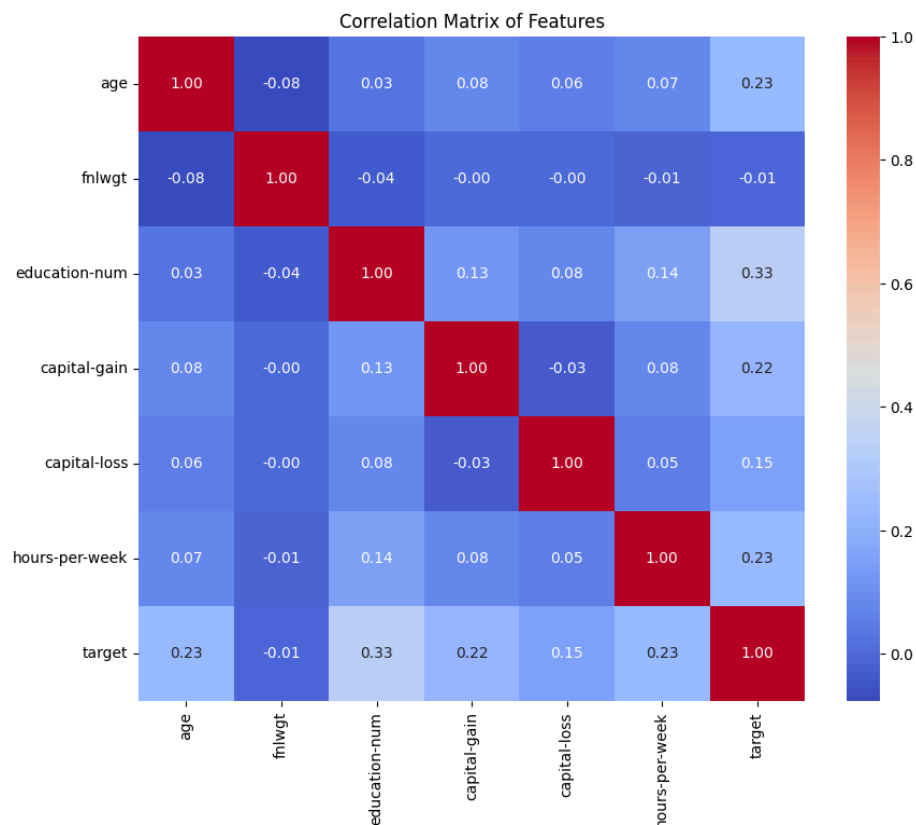
Introduction:

EDA is the process of investigating your dataset to discover patterns, spot anomalies, test hypotheses, and check your underlying assumptions. Instead of immediately jumping into complex machine learning models, EDA relies on summarizing the main characteristics of the data, usually through visual methods like graphs, plots, and correlation matrices. Some significance of EDA:

- Instant Pattern Recognition
- Simplifying Multi-Dimensional Data:
- Spotting Data Quality Issues Safely
- Guiding the Modeling Strategy

Correlation Matrix:

```
# EDA:
# Visualization: Correlation with target
plt.figure(figsize=(10, 8))
X_target=X
X_target['target']=y['income']
corr_matrix=X_target.corr(numeric_only=True)
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix of Features')
```

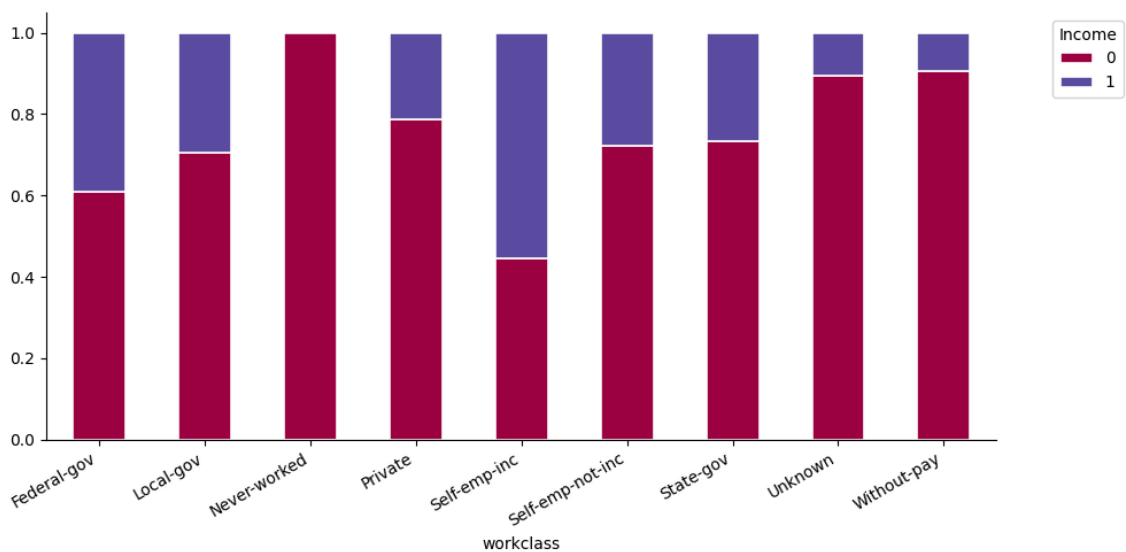


Explanation:

- The correlation matrix represents the relation between 2 numerical features.
- Target value is preprocessed to integer to make correlation with other numerical features.
- From the matrix, target value is slightly positively correlated with the 'education-num' features and little negatively correlated with the 'fnlwgt-final weight'.
- We can identify the strong and weak relation between 2 numerical features.

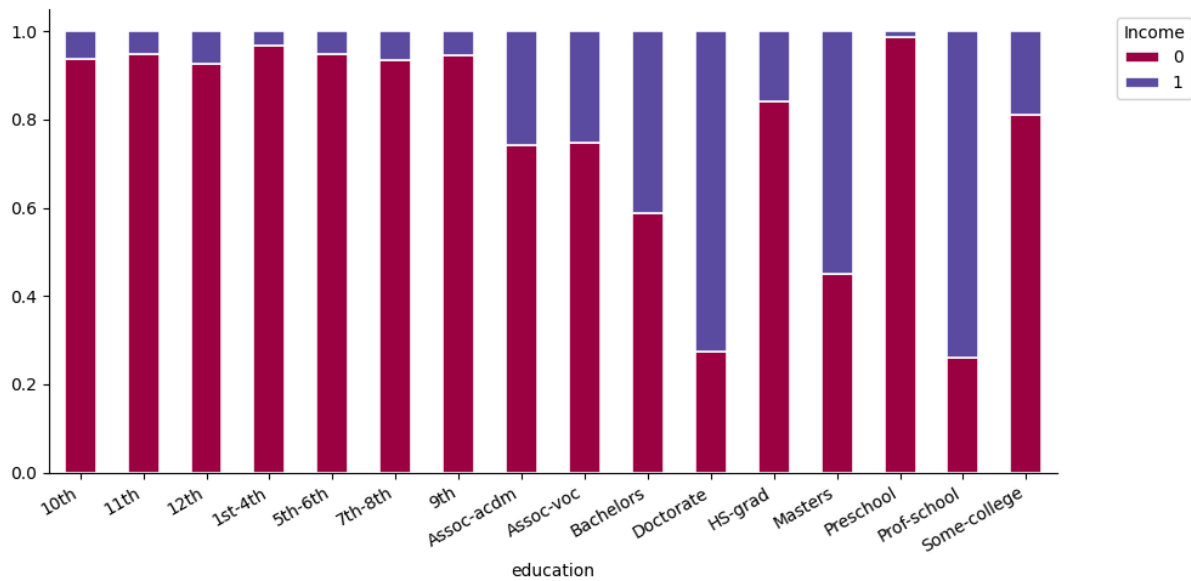
Crosstab Representation:

```
# EDA
# Crosstab representation:
for c in categorical_columns:
    ct = pd.crosstab(X[c], y['income'], normalize='index')
    ax = ct.plot(kind='bar', stacked=True, figsize=(10, 5), colormap='Spectral',
                edgecolor='white', linewidth=1.2)
    plt.xticks(rotation=30, ha='right')
    plt.legend(title='Income', bbox_to_anchor=(1.05, 1), loc='upper left')
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    plt.tight_layout()
    plt.show()
```



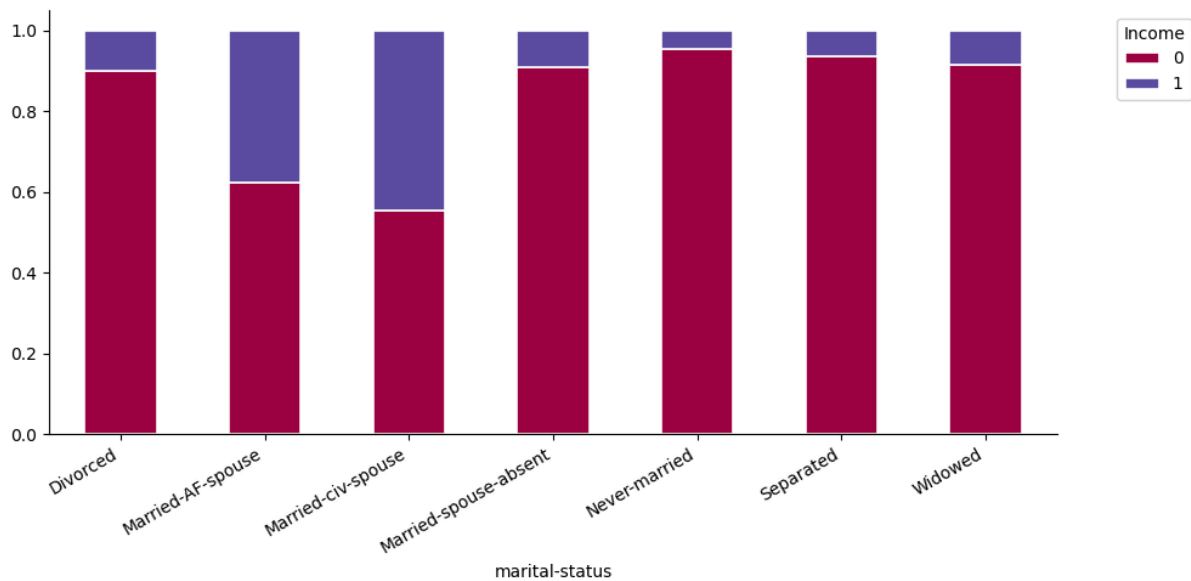
Explanation:

- This crosstab represent which unique item of 'workclass' falls in which target class.
- Here 'never-worked' people income in '0' class
- 'Self-emp-inc' is slightly dominated to put the record in '1' class.



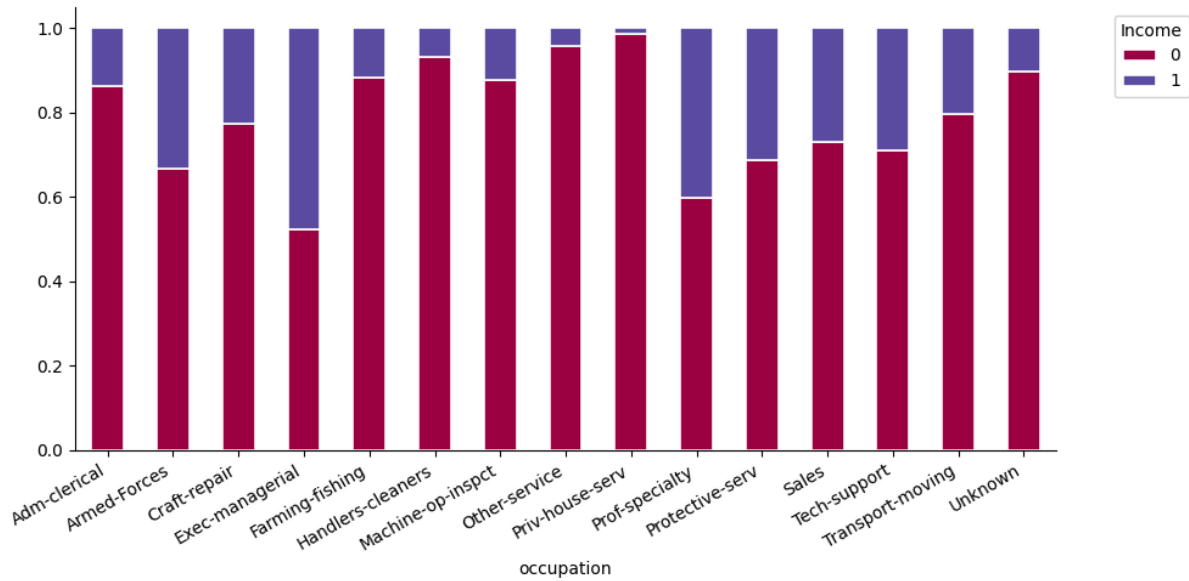
Explanation:

- This crosstab represent which unique item of 'education' falls in which target class.
- Here '1st-4th' class people dominates income in '0' class
- 'Doctorate' & 'prof-school' is dominated to put the record in '1' class.



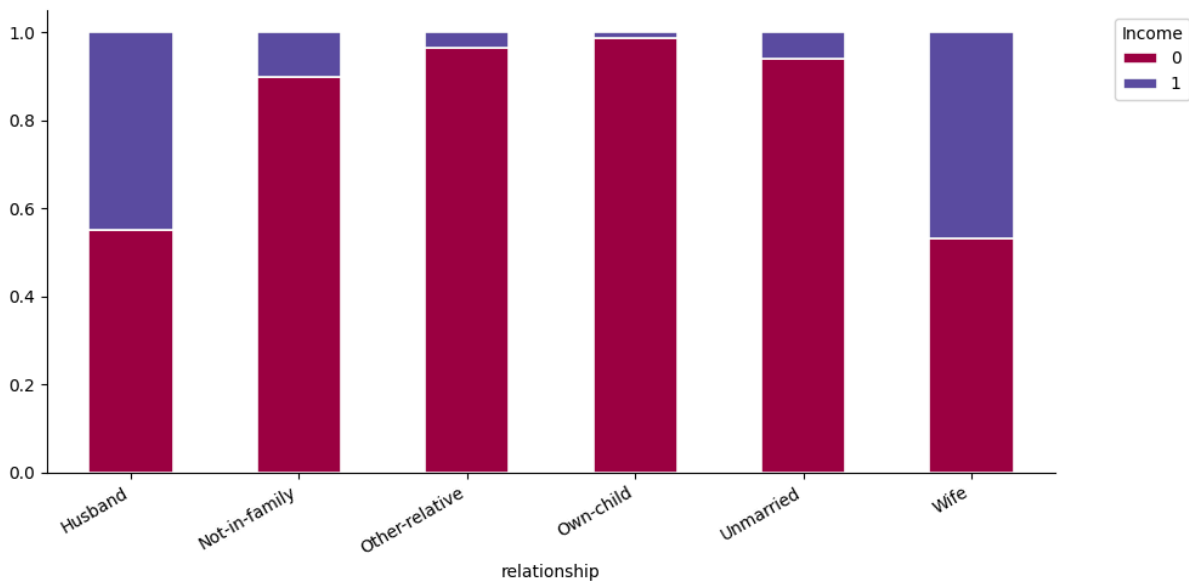
Explanation:

- This crosstab represent which unique item of 'marital-status' falls in which target class.
- Here 'never-married' & 'Divorced' class people dominate income in '0' class
- 'married-civ-spouse' is dominated to put the record in '1' class.



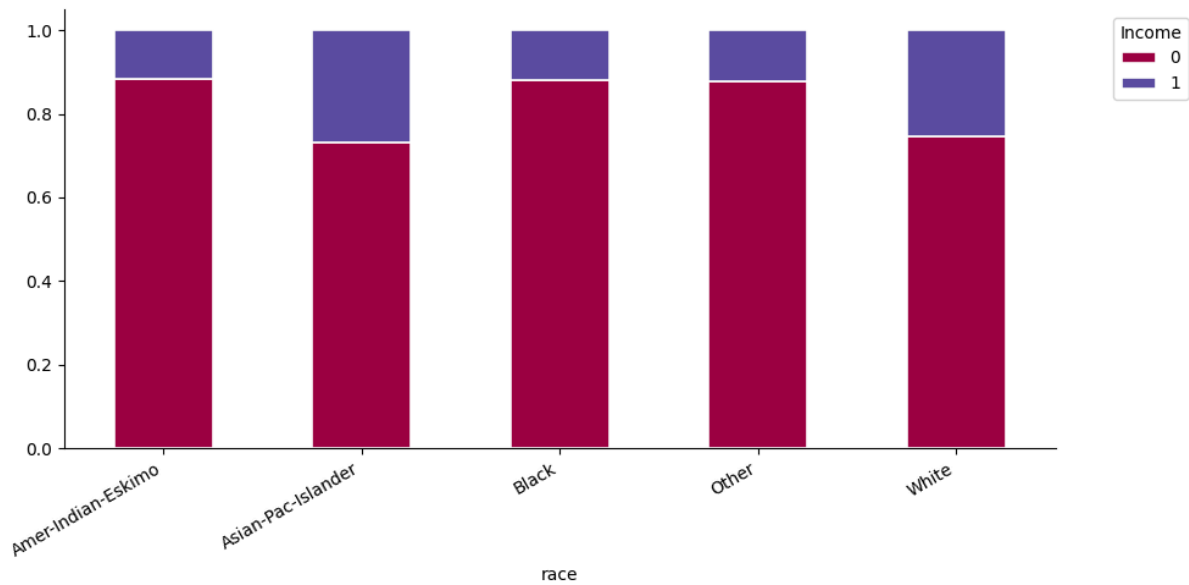
Explanation:

- This crosstab represent which unique item of 'occupation' falls in which target class.
- High income '0' → 'priv-house-serv'
- High income '1' → 'exec-managerial'



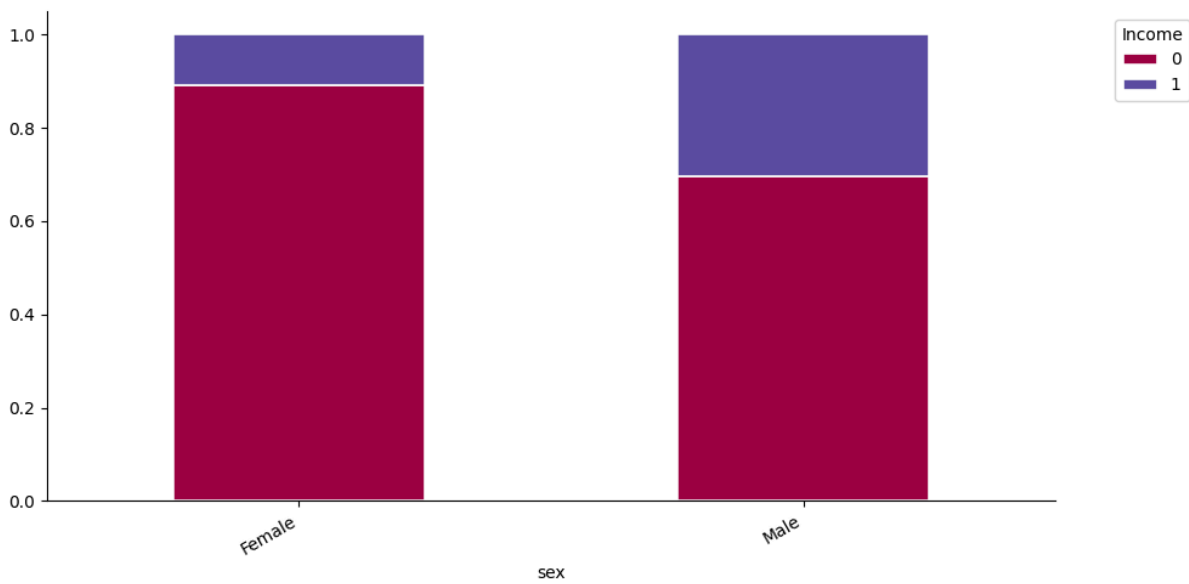
Explanation:

- This crosstab represent which unique item of 'relationship' falls in which target class.
- High income '0' → 'own-child'
- High income '1' → 'Husband' 'Wife'



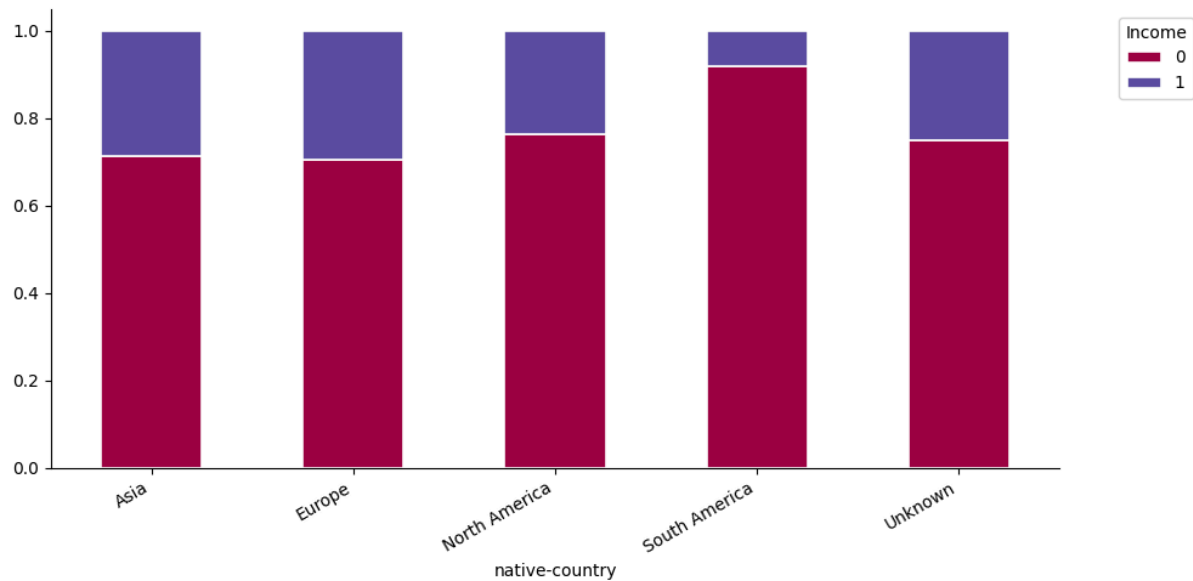
Explanation:

- This crosstab represent which unique item of 'race' falls in which target class.
- High income '0' → 'black' 'other'
- High income '1' → 'Asian-Pac-Islander'



Explanation:

- This crosstab represent which unique item of 'sex' falls in which target class.
- High income '0' → 'Female'
- High income '1' → 'Male'



Explanation:

- This crosstab represent which unique item of 'native-country' falls in which target class.
- High income '0' → 'South America'
- High income '1' → 'North America' 'Unknown'

Dataset Description After Preprocessing:

	age	fnlwgt	education-num	capital-gain	capital-loss	hours-per-week
count	48842.000000	4.884200e+04	48842.000000	48842.000000	48842.000000	48842.000000
mean	38.643585	1.896641e+05	10.078089	1079.067626	87.502314	40.422382
std	13.710510	1.056040e+05	2.570973	7452.019058	403.004552	12.391444
min	17.000000	1.228500e+04	1.000000	0.000000	0.000000	1.000000
25%	28.000000	1.175505e+05	9.000000	0.000000	0.000000	40.000000
50%	37.000000	1.781445e+05	10.000000	0.000000	0.000000	40.000000
75%	48.000000	2.376420e+05	12.000000	0.000000	0.000000	45.000000
max	90.000000	1.490400e+06	16.000000	99999.000000	4356.000000	99.000000

Explanation:

- From this, it express the statistics analysis of the numerical features of the dataset along with target value.
- The parameters are → mean, standard deviation, min, Different Quantile ranges, max

Discussion:

- From this experiment I have known about some visualization methods to display data like correlation matrix, crosstab representation.
- Get the description of the statistical analysis.

Experiment-3: Mining Frequent Patterns and Association Rules

Introduction:

In the era of big data, organizations across industries collect vast quantities of transactional, relational, and demographic data. Extracting actionable knowledge from these massive repositories requires advanced data mining methodologies that go beyond simple statistical summaries. Among these methodologies, **Frequent Pattern Mining (FPM)** and **Association Rule Mining (ARM)** stand out as fundamental techniques designed to uncover hidden dependencies, correlations, and co-occurrence structures among attributes within a dataset.

- **Support**→The absolute frequency of an itemset 'X→Y' in the database. 'T' is the total number of transactions. The relative frequency, known as the support, is the probability that a transaction contains X:

$$\text{Support}(X \rightarrow Y), S = \frac{\sigma(X \cup Y)}{|T|}$$

- **Confidence**→ It measures the conditional probability of the occurrence of the consequent Y, given the occurrence of the antecedent X. It assesses the operational reliability of the generated rule.

$$\text{Confidence}(X \rightarrow Y) = P(Y | X) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

- **Lift**→ It evaluates the strength of a rule by measuring the ratio of the observed joint support of X and Y to the expected support if X and Y were statistically independent.

$$\text{Lift}(X \rightarrow Y) = \frac{P(X \cup Y)}{P(X) \times P(Y)}$$

The interpretation of Lift values is defined as follows:

- **Lift = 1:** X and Y are independent. The rule provides no predictive value.
- **Lift > 1:** X and Y are positively correlated. The occurrence of X significantly increases the likelihood of Y.
- **Lift < 1:** X and Y are negatively correlated, indicating that the presence of X discourages the presence of Y.

- **Apriori Algorithm**→ Which states that any subset of a frequent itemset must also be frequent. If an itemset is infrequent, all its supersets are immediately pruned, drastically reducing the search space.
- **FP-Growth Algorithm**→ Compresses the transactional database into a highly optimized, frequent-pattern tree structure (FP-Tree). This eliminates the expensive bottleneck of repeatedly scanning the database and avoids explicit candidate generation by mining the tree recursively via conditional patterns.

Dataset Preprocessing for mining frequent patterns:

```
# Frequent Pattern Analysis and Apriori Algorithm:
yy=y['income'].replace({1:'G50K', 0:'LE50K'})
df['income']=yy
df=df.drop(columns=['fnlwgt','education-num','capital-gain','capital-loss','hours-per-week'])
def Age(x):
    if x<=18:
        return 'Child'
    elif x>18 and x<=25:
        return 'Youth'
    elif x>25 and x<40:
        return 'Adult'
    else:
        return 'Senior'

# Age Bining
df['age']=df['age'].apply(Age)
df.head()
```

	age	workclass	education	marital-status	occupation	relationship	race	sex	native-country	income
0	Adult	State-gov	Bachelors	Never-married	Adm-clerical	Not-in-family	White	Male	North America	LE50K
1	Senior	Self-emp-not-inc	Bachelors	Married-civ-spouse	Exec-managerial	Husband	White	Male	North America	LE50K
2	Adult	Private	HS-grad	Divorced	Handlers-cleaners	Not-in-family	White	Male	North America	LE50K
3	Senior	Private	11th	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	North America	LE50K
4	Adult	Private	Bachelors	Married-civ-spouse	Prof-specialty	Wife	Black	Female	North America	LE50K

Explanation:

- ‘Age’ is binning with string value for pattern analysis.

Transaction Dataset Creation:

```
import random
random.seed(42)

dataset=[]
col=list(df.columns)

for i in range(len(df)):
    temp=[]
    random_col=random.choices(col,k=random.randint(2,len(col))) # random column choice
    random_col=list(set(random_col)) # unique column
    for j in random_col:
        temp.append(df.iloc[i][j])
    dataset.append(temp)

dataset=[[str(i) for i in row] for row in dataset]

for i in range(10):
    print(dataset[i],end='\n\n')
```

```
['Adult', 'Bachelors']

['North America', 'White', 'Senior']

['Male', 'White', 'HS-grad', 'Handlers-cleaners', 'Adult', 'Not-in-family']

['Married-civ-spouse', 'North America', 'LE50K', 'Black', 'Private', 'Husband', 'Senior']

['Married-civ-spouse']

['Female', 'Married-civ-spouse', 'North America', 'LE50K', 'Wife']

['Not-in-family', 'Female', '9th', 'Senior']

['North America', 'Married-civ-spouse', 'Exec-managerial']

['Masters', 'G50K', 'Never-married']

['Male', 'White', 'Private']
```

Explanation:

- Random values are chosen to create the transaction and set operation for uniqueness.

Built Transaction Encoder:

```
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules, fpgrowth

te=TransactionEncoder()
te_ary=te.fit(dataset).transform(dataset)
new_df=pd.DataFrame(te_ary, columns=te.columns_)

new_df.head()
```

	10th	11th	12th	1st-4th	5th-6th	7th-8th	9th	Adm-clerical	Adult	Amer-Indian-Eskimo	...	State-gov	Tech-support	Transport-moving	Unknown	Unmarried	White	Widowed	Wife	Without-pay	Youth
0	False	False	False	False	False	False	False	False	True	False	...	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	True	False	False	False	False
2	False	False	False	False	False	False	False	False	True	False	...	False	False	False	False	False	True	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False

5 rows x 69 columns

Explanation:

- Transaction encoder converts each transaction into a boolean matrix.
- It tells which itemsets are present in a specific transaction.

Apriori Frequent Itemset with minimum support 0.05:

```
frequent_itemsets=apriori(new_df, min_support=0.05, use_colnames=True)
print(frequent_itemsets.sort_values(by='support', ascending=False).to_string(index=False))
print(len(frequent_itemsets))
```

support	itemsets
0.420253	(North America)
0.389030	(White)
0.340322	(LE50K)
0.322427	(Private)
0.295238	(Male)
0.206339	(Married-civ-spouse)
0.196286	(Senior)
0.179047	(Husband)
0.169731	(White, North America)
0.163486	(Adult)
0.149380	(Never-married)
0.148315	(Female)
0.145326	(LE50K, North America)
0.144200	(HS-grad)
0.138631	(Private, North America)
0.132366	(White, LE50K)
0.126510	(White, Private)
0.126326	(Male, North America)
0.118955	(White, Male)
0.116191	(Not-in-family)
0.113857	(Private, LE50K)
0.107244	(G50K)
0.098112	(Some-college)
0.095737	(Private, Male)
...	
0.051247	(LE50K, Married-civ-spouse)
0.050510	(Private, Female)
0.050018	(HS-grad, Private)

69

Explanation:

- Based on support determines the itemset which are greater equal than support.

Rule generation using Apriori based frequent itemset:

```
rules=association_rules(frequent_itemsets, metric="confidence", min_threshold=0.2)
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
print(f"Rules generated by Apriori: {len(rules)}")
```

	antecedents	consequents	support	confidence	lift
0	(Adult)	(LE50K)	0.057246	0.350157	1.028898
1	(Adult)	(Male)	0.051636	0.315842	1.069790
2	(Adult)	(North America)	0.070247	0.429681	1.022433
3	(Adult)	(Private)	0.058003	0.354790	1.100372
4	(Adult)	(White)	0.063675	0.389480	1.001158
..	...	...	...	...	...
65	(White, North America)	(Male)	0.052332	0.308323	1.044322
66	(Male, North America)	(White)	0.052332	0.414263	1.064860
67	(White, Private)	(North America)	0.056284	0.444894	1.058634
68	(White, North America)	(Private)	0.056284	0.331604	1.028462
69	(Private, North America)	(White)	0.056284	0.405996	1.043612

[70 rows x 5 columns]
Rules generated by Apriori: 70

Explanation:

- Based on the itemset, different kinds of strong association rule generated using 'confidence' value.

FP-Growth Frequent Itemset with minimum support 0.05:

```
frequent_fp=fpgrowth(new_df, min_support=0.05, use_colnames=True)
rules_fp=association_rules(frequent_fp, metric="lift",min_threshold=1.0)
print(rules_fp[['antecedents','consequents','support','confidence','lift']])
print(f"Rules generated by FP-Growth: {len(rules_fp)}")
```

	antecedents	consequents	support	confidence	lift
0	(White)	(Adult)	0.063675	0.163676	1.001158
1	(Adult)	(White)	0.063675	0.389480	1.001158
2	(Male)	(Adult)	0.051636	0.174896	1.069790
3	(Adult)	(Male)	0.051636	0.315842	1.069790
4	(Private)	(Adult)	0.058003	0.179896	1.100372
..	...	...	...	...	...
81	(LE50K)	(Never-married)	0.066091	0.194200	1.300046
82	(Never-married)	(Private)	0.054318	0.363624	1.127770
83	(Private)	(Never-married)	0.054318	0.168466	1.127770
84	(Never-married)	(North America)	0.063634	0.425987	1.013644
85	(North America)	(Never-married)	0.063634	0.151418	1.013644

[86 rows x 5 columns]
Rules generated by FP-Growth: 86

Explanation:

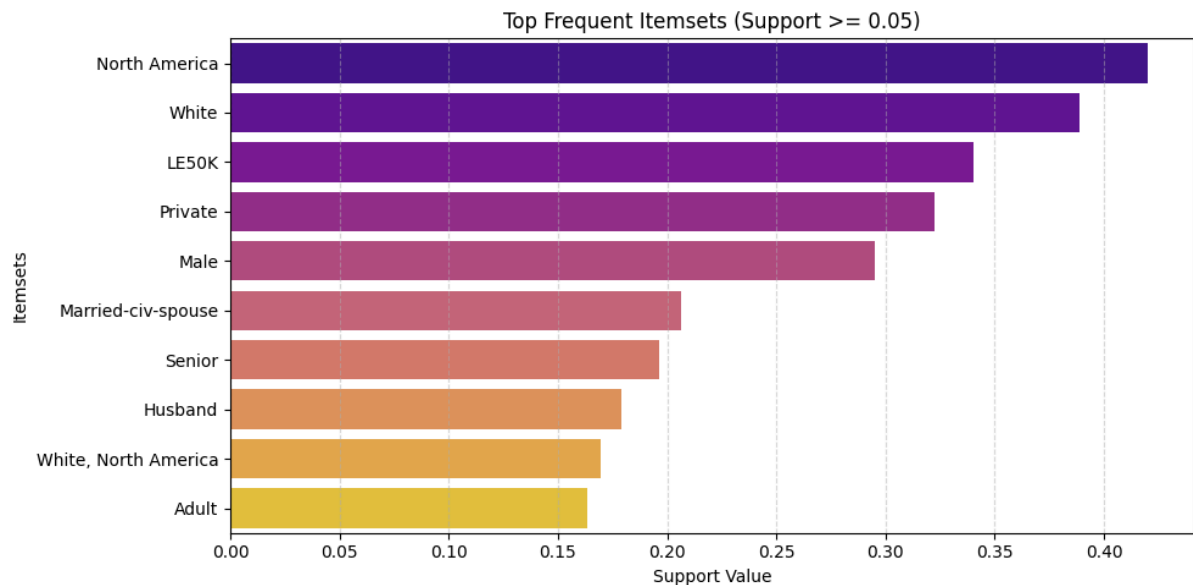
- Based on the itemset, different kinds of strong association rule generated using 'lift' value for fp-growth.

Top 10 frequent itemset:

```
top_10_itemsets=frequent_itemsets.sort_values(by='support', ascending=False).head(10).copy()

# Convert itemsets frozenset type to string strings for clean plotting labels
top_10_itemsets['itemsets'] = top_10_itemsets['itemsets'].apply(lambda x: ', '.join(list(x)))

# Plotting
plt.figure(figsize=(10, 5))
sns.barplot(x='support', y='itemsets', data=top_10_itemsets, palette='plasma')
plt.title('Top Frequent Itemsets (Support >= 0.05)')
plt.xlabel('Support Value')
plt.ylabel('Itemsets')
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



Discussion:

- From this experiment I have learned the parameters of interestingness measurement.
- Create transactions.
- Build new transactions using association rule mining and apply them to 2 different algorithms.

Experiment-4: Application of Classification techniques

Introduction:

Classification is the cornerstone of supervised learning. It is the process of training an algorithm to map input features to a specific, discrete output or the target label. It is used to predict categorical class level. Through this we construct a model based on the training set and the class labels (the values in a classifying attribute) and use it in classifying new data.

Types of Classification Tasks

1. **Binary Classification:** The target variable has exactly two possible outcomes. This is the most common type.
 - *Example:* Predicting if an email is **Spam (1)** or **Not Spam (0)**.
2. **Multiclass Classification:** The target variable has three or more possible outcomes, and each data point belongs to exactly one class.
 - *Example:* Hand Written Digit Recognition (0-9)

Some types of Classification Techniques:

1. Logistic Regression
2. Naive Bayes
3. Decision Tree

Some Terminology using in Classification:

1. Confusion Matrix:

	Predicted	
Actual	TP (True Positive) (Correct Detection)	FN (False Negative) (Missed Once)
	FP (False Positive) (False Alarm)	TN (True Negative) (Correct Rejection)

2. Accuracy Score: Percentage of test set tuples that are correctly classified.

$$\text{Accuracy} = \frac{TP+TN}{\text{Total}}$$

3. Precision Score: **Exactness**. what % of tuples that the classifier labeled as positive are actually positive

$$\text{precision} = \frac{TP}{TP+FP}$$

4. Recall: Completeness: what % of positive tuples did the classifier label as positive

$$\text{Recall} = \frac{TP}{TP+FN}$$

5. F1 Score: Harmonic mean of precision and recall

$$F1 = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

Summary of the metrics

	Predicted		
Actual	TP	FN	$\text{Recall} = \frac{TP}{TP+FN}$
	FP	TN	$\text{specificity} = \frac{TN}{FP+TN}$
	$\text{precision} = \frac{TP}{TP+FP}$	$\text{NPV} = \frac{TN}{FN+TN}$	$\text{Accuracy} = \frac{TP+TN}{\text{Total}}$

Sensitivity → Recall

Specificity → True negative recognition rate

NPV → Negative Predicted Value

Precision → PPV → Positive Predicted Value

Logistic Regression:

- A classification algorithm
- Mostly used in Binary Classification
- The formula is $z = w \cdot x + b$, here x is input values and w is weight that is learned based on iteration using learning rate and gradient descent function.
- Learning rate → what amount of steps should forward or backward to converge to global minima.
- Gradient Descent → Assign new weights by adjusting the old weight value using loss function and learning rate.

$$w_{\text{new}} = w_{\text{old}} - (\text{Learning Rate} \times \text{Gradient})$$

- After completing weight calculation, predict the z value and pass it to the sigmoid function to get the probability.

$$y_{\text{hat}} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- Using a threshold value, the model helps to classify the output properly.

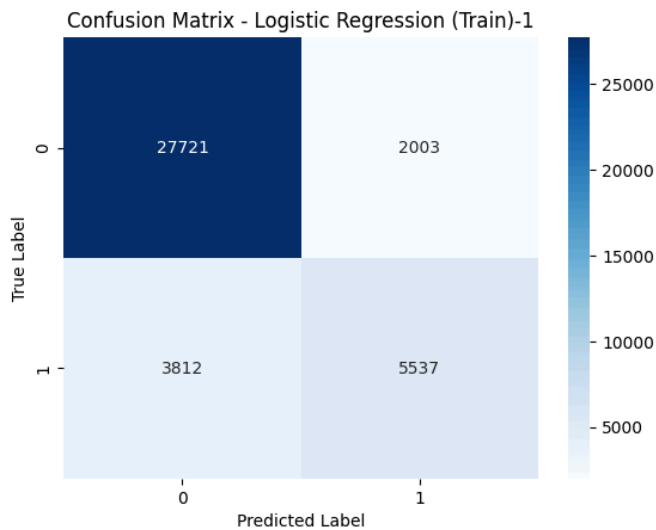
For test_size=0.2

```
# Train Test Splitting: 0.2
X_train,X_test,y_train,y_test=train_test_split(X_encoded,y,test_size=0.2,random_state=42,stratify=y)

# Logistics Regression:
log_reg=LogisticRegression()
log_reg.fit(X_train,y_train)
```

```
y_pred=log_reg.predict(X_train)
cm=confusion_matrix(y_train,y_pred)
print("Training Portion:")
print("Accuracy->",np.round(accuracy_score(y_train,y_pred),2))
print("Precision->",np.round(precision_score(y_train,y_pred),2))
print("Recall->",np.round(recall_score(y_train,y_pred),2))
print("F1-score->",np.round(f1_score(y_train,y_pred),2))

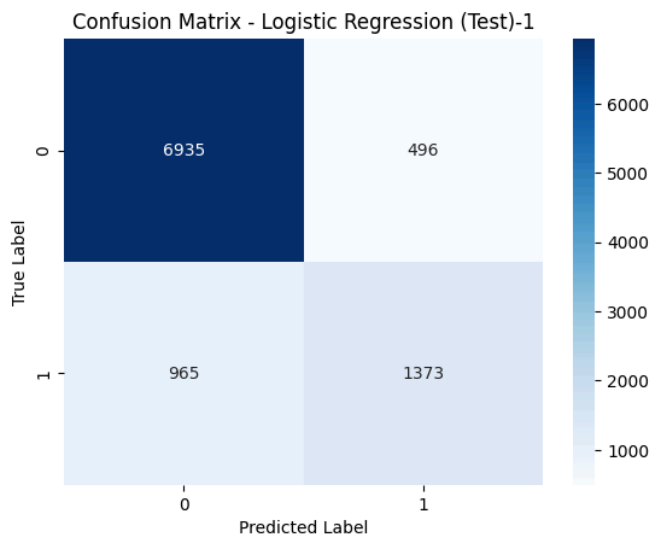
sns.heatmap(cm, annot=True, cmap='Blues', fmt='g')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Logistic Regression (Train)-1')
plt.show()
```



Training Portion:
 Accuracy-> 0.85
 Precision-> 0.73
 Recall-> 0.59
 F1-score-> 0.66

```
y_pred=log_reg.predict(X_test)
cm=confusion_matrix(y_test,y_pred)
print("Testing Portion")
print("Accuracy->",np.round(accuracy_score(y_test,y_pred),2))
print("Precision->",np.round(precision_score(y_test,y_pred),2))
print("Recall->",np.round(recall_score(y_test,y_pred),2))
print("F1-score->",np.round(f1_score(y_test,y_pred),2))

sns.heatmap(cm, annot=True, cmap='Blues', fmt='g')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Logistic Regression (Test)-1')
plt.show()
```



Testing Portion
 Accuracy-> 0.85
 Precision-> 0.73
 Recall-> 0.59
 F1-score-> 0.65

Test size=0.3

```
# Train Test Splitting: 0.3
X_train,X_test,y_train,y_test=train_test_split(X_encoded,y,test_size=0.3,random_state=42,stratify=y)

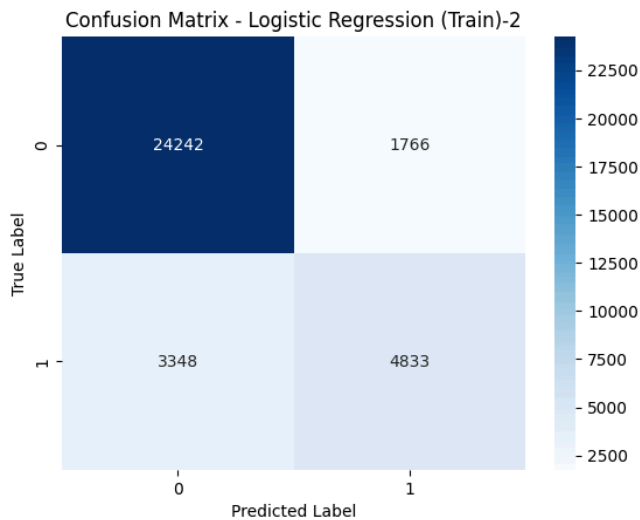
# Logistics Regression:
log_reg=LogisticRegression()
log_reg.fit(X_train,y_train)
```

```

y_pred=log_reg.predict(X_train)
cm=confusion_matrix(y_train,y_pred)
print("Training Portion:")
print("Accuracy->",np.round(accuracy_score(y_train,y_pred),2))
print("Precision->",np.round(precision_score(y_train,y_pred),2))
print("Recall->",np.round(recall_score(y_train,y_pred),2))
print("F1-score->",np.round(f1_score(y_train,y_pred),2))

sns.heatmap(cm, annot=True, cmap='Blues', fmt='g')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Logistic Regression (Train)-2')
plt.show()

```



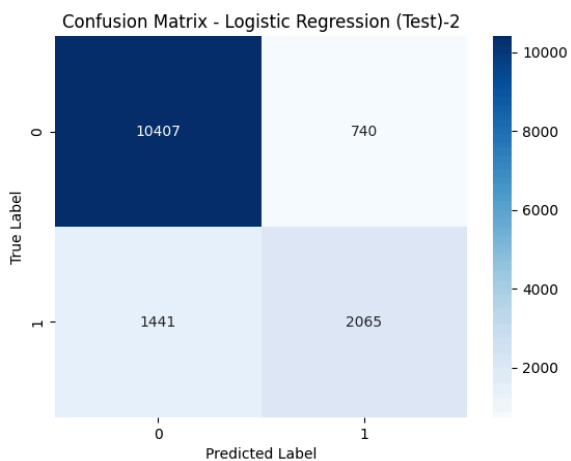
Training Portion:
 Accuracy-> 0.85
 Precision-> 0.73
 Recall-> 0.59
 F1-score-> 0.65

```

y_pred=log_reg.predict(X_test)
cm=confusion_matrix(y_test,y_pred)
print("Testing Portion")
print("Accuracy->",np.round(accuracy_score(y_test,y_pred),2))
print("Precision->",np.round(precision_score(y_test,y_pred),2))
print("Recall->",np.round(recall_score(y_test,y_pred),2))
print("F1-score->",np.round(f1_score(y_test,y_pred),2))

sns.heatmap(cm, annot=True, cmap='Blues', fmt='g')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Logistic Regression (Test)-2')
plt.show()

```



Testing Portion
 Accuracy-> 0.85
 Precision-> 0.74
 Recall-> 0.59
 F1-score-> 0.65

Naive Bayes Classifier:

- Based on Bayes Theorem.
- we can derive a Bayes Classifier to compute the posterior probability of classifying an object X to a class C.

$$P(x | C) = \frac{P(x_1|C) * P(x_2|C) * \dots * P(x_n|C)}{P(x)}$$
$$P(x | C) \propto \prod_{i=1}^n P(x_i | C)$$

- The classifier considers every feature as an independent feature. None is dependent on others.
- At last given the output which has a high probability value.

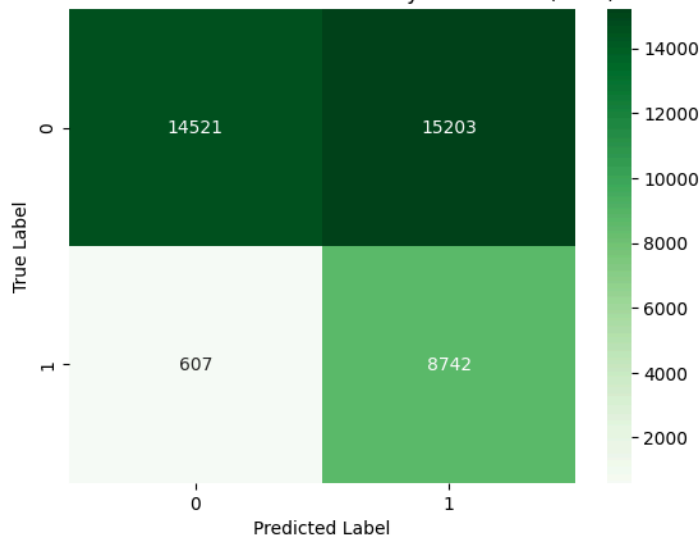
Test size=0.2

```
# Train Test Splitting: 0.2
X_train,X_test,y_train,y_test=train_test_split(X_encoded,y,test_size=0.2,random_state=42,stratify=y)

# Naive Bayes Classifier
gnb=GaussianNB()
gnb.fit(X_train,y_train)
y_pred=gnb.predict(X_train)
cm=confusion_matrix(y_train,y_pred)
print("Training Portion:")
print("Accuracy->",np.round(accuracy_score(y_train,y_pred),2))
print("Precision->",np.round(precision_score(y_train,y_pred),2))
print("Recall->",np.round(recall_score(y_train,y_pred),2))
print("F1-score->",np.round(f1_score(y_train,y_pred),2))

sns.heatmap(cm, annot=True, cmap='Greens', fmt='g')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Gaussian Naive Bayes Classifier (Train)-1')
plt.show()
```

Confusion Matrix - Gaussian Naive Bayes Classifier (Train)-1



Training Portion:
Accuracy-> 0.6
Precision-> 0.37
Recall-> 0.94
F1-score-> 0.53

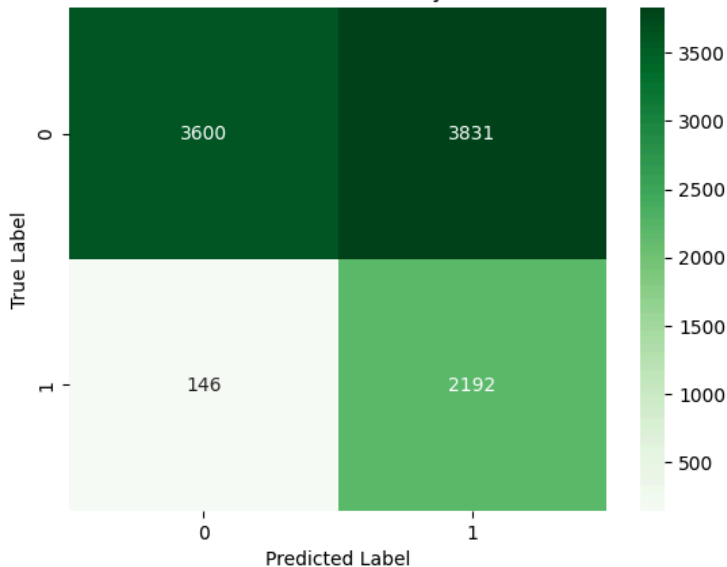
```

y_pred=gnb.predict(X_test)
cm=confusion_matrix(y_test,y_pred)
print("Testing Portion")
print("Accuracy->",np.round(accuracy_score(y_test,y_pred),2))
print("Precision->",np.round(precision_score(y_test,y_pred),2))
print("Recall->",np.round(recall_score(y_test,y_pred),2))
print("F1-score->",np.round(f1_score(y_test,y_pred),2))

sns.heatmap(cm, annot=True, cmap='Greens', fmt='g')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Gaussian Naive Bayes Classifier (Test)-1')
plt.show()

```

Confusion Matrix - Gaussian Naive Bayes Classifier (Test)-1



Testing Portion
 Accuracy-> 0.59
 Precision-> 0.36
 Recall-> 0.94
 F1-score-> 0.52

Test size=0.3

```

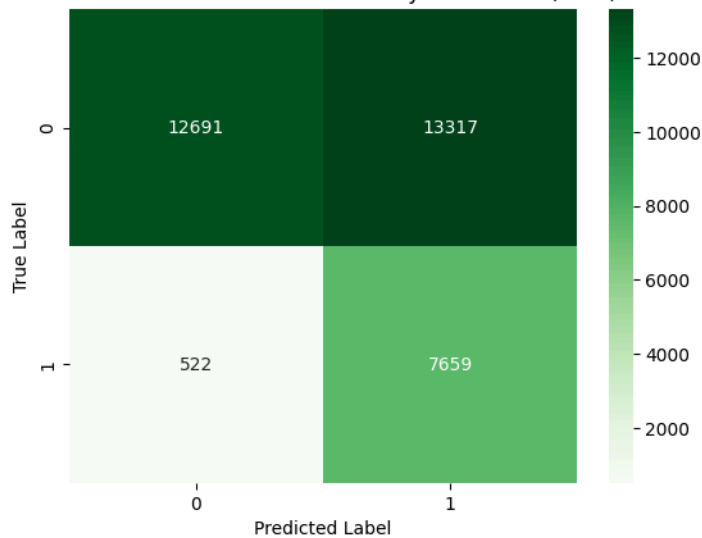
# Train Test Splitting: 0.3
X_train,X_test,y_train,y_test=train_test_split(X_encoded,y,test_size=0.3,random_state=42,stratify=y)

# Naive Bayes Classifier
gnb=GaussianNB()
gnb.fit(X_train,y_train)
y_pred=gnb.predict(X_train)
cm=confusion_matrix(y_train,y_pred)
print("Training Portion:")
print("Accuracy->",np.round(accuracy_score(y_train,y_pred),2))
print("Precision->",np.round(precision_score(y_train,y_pred),2))
print("Recall->",np.round(recall_score(y_train,y_pred),2))
print("F1-score->",np.round(f1_score(y_train,y_pred),2))

sns.heatmap(cm, annot=True, cmap='Greens', fmt='g')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Gaussian Naive Bayes Classifier (Train)-2')
plt.show()

```

Confusion Matrix - Gaussian Naive Bayes Classifier (Train)-2

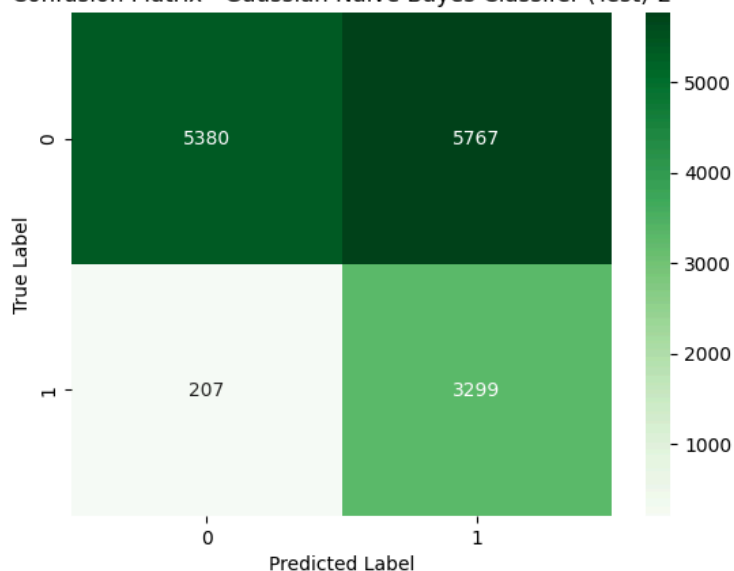


Training Portion:
 Accuracy-> 0.6
 Precision-> 0.37
 Recall-> 0.94
 F1-score-> 0.53

```
y_pred=gnb.predict(X_test)
cm=confusion_matrix(y_test,y_pred)
print("Testing Portion")
print("Accuracy->",np.round(accuracy_score(y_test,y_pred),2))
print("Precision->",np.round(precision_score(y_test,y_pred),2))
print("Recall->",np.round(recall_score(y_test,y_pred),2))
print("F1-score->",np.round(f1_score(y_test,y_pred),2))

sns.heatmap(cm, annot=True, cmap='Greens', fmt='g')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Gaussian Naive Bayes Classifier (Test)-2')
plt.show()
```

Confusion Matrix - Gaussian Naive Bayes Classifier (Test)-2



Testing Portion
 Accuracy-> 0.59
 Precision-> 0.36
 Recall-> 0.94
 F1-score-> 0.52

Decision Tree Classifier:

- A classification model that uses trees (node and branch) to predict values.
- Tree is constructed in a top-down, recursive, divide-and-conquer manner
- Examples are partitioned recursively based on selected attributes
- On each node, attributes are selected based on the training examples on that node, and a heuristic or statistical measure (information gain, Gini index)

Entropy Approach	Gini Approach
• $H(Y) = - \sum_{i=1}^m p_i \log(P_i)$ $m \rightarrow$ class. • Then calculate the average entropy of the branch and entropy of the node also. • After calculate information gain (IG). • $IG = \text{Entropy}(\text{node}) - \text{Weighted entropy of branches}$. • Select the high IG as a node of the decision tree.	• $\text{Gini} = 1 - \sum_{i=1}^m p_i^2$ $m \rightarrow$ class. • Calculate the gini value of nodes and weighted gini value of branches. • Take the difference between them to select the actual node for tree making.

Criterion- 'Entropy'

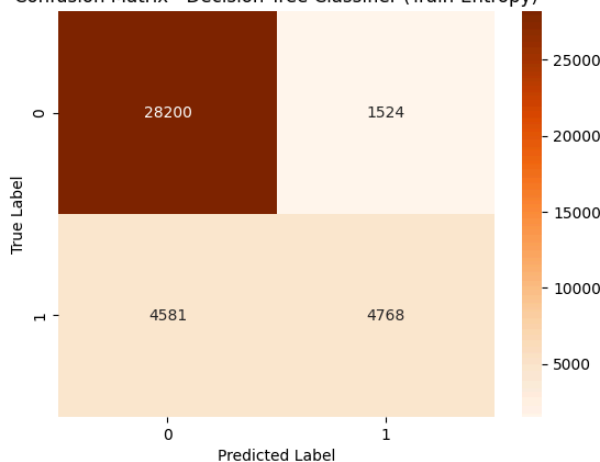
```
# Train Test Splitting: 0.2
X_train,X_test,y_train,y_test=train_test_split(X_encoded,y,test_size=0.2,random_state=42, stratify=y)

# Decision Tree classifier

dt=DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=0)
dt.fit(X_train,y_train)
y_pred=dt.predict(X_train)
cm=confusion_matrix(y_train,y_pred)
print("Training Portion:")
print("Accuracy->",np.round(accuracy_score(y_train,y_pred),2))
print("Precision->",np.round(precision_score(y_train,y_pred),2))
print("Recall->",np.round(recall_score(y_train,y_pred),2))
print("F1-score->",np.round(f1_score(y_train,y_pred),2))

sns.heatmap(cm, annot=True, cmap='Oranges', fmt='g')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Decision Tree Classifier (Train-Entropy)')
plt.show()
```

Confusion Matrix - Decision Tree Classifier (Train-Entropy)



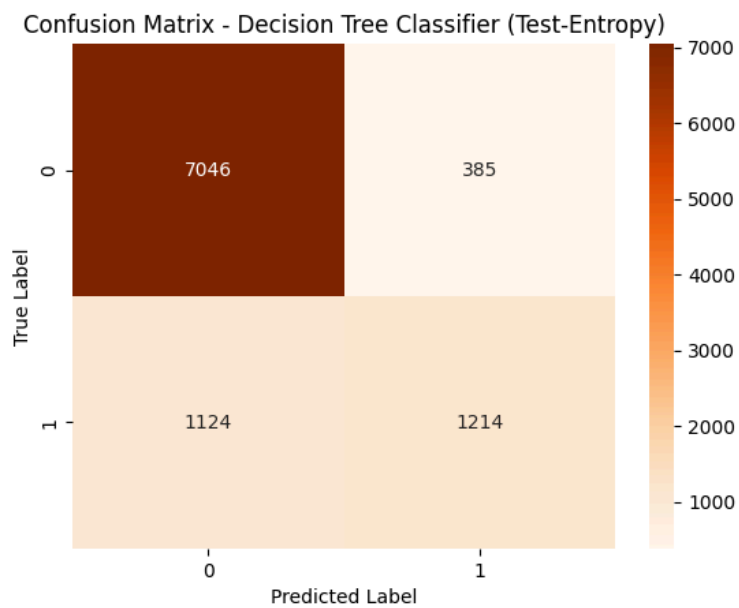
Training Portion:
Accuracy-> 0.84
Precision-> 0.76
Recall-> 0.51
F1-score-> 0.61

```

y_pred=dt.predict(X_test)
cm=confusion_matrix(y_test,y_pred)
print("Testing Portion")
print("Accuracy->",np.round(accuracy_score(y_test,y_pred),2))
print("Precision->",np.round(precision_score(y_test,y_pred),2))
print("Recall->",np.round(recall_score(y_test,y_pred),2))
print("F1-score->",np.round(f1_score(y_test,y_pred),2))

sns.heatmap(cm, annot=True, cmap='Oranges', fmt='g')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Decision Tree Classifier (Test-Entropy)')
plt.show()

```



Testing Portion
 Accuracy-> 0.85
 Precision-> 0.76
 Recall-> 0.52
 F1-score-> 0.62

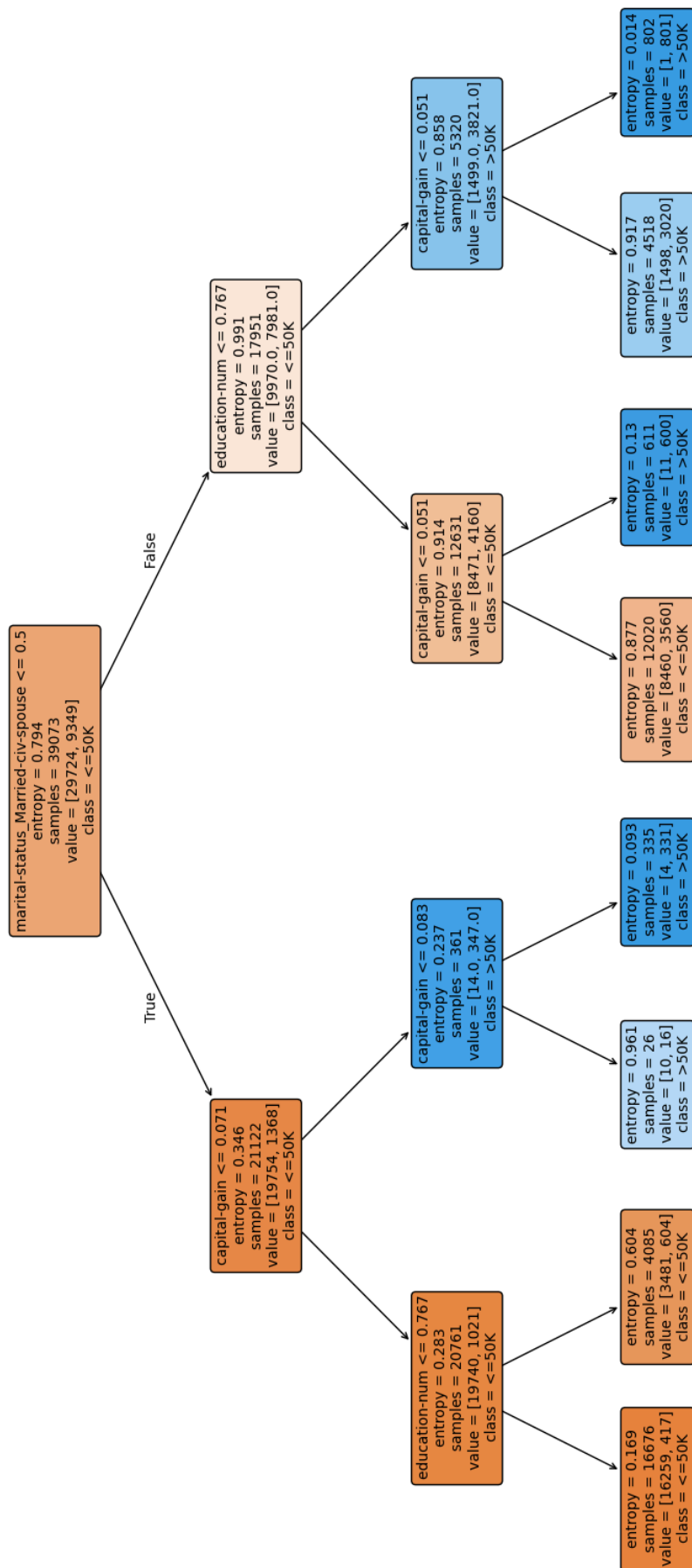
```

plt.figure(figsize=(20, 10))

plot_tree(dt,
           feature_names=X_encoded.columns,
           class_names=['<=50K', '>50K'],
           filled=True,
           rounded=True,
           fontsize=10)

plt.show()

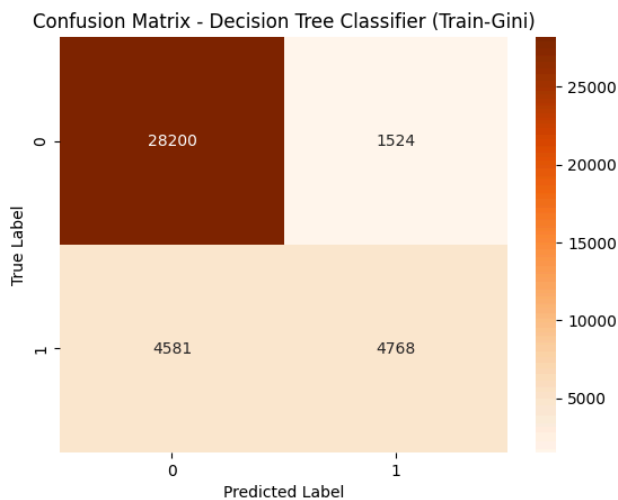
```



Criterion- 'gini'

```
dt=DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=0)
dt.fit(X_train,y_train)
y_pred=dt.predict(X_train)
cm=confusion_matrix(y_train,y_pred)
print("Training Portion:")
print("Accuracy->",np.round(accuracy_score(y_train,y_pred),2))
print("Precision->",np.round(precision_score(y_train,y_pred),2))
print("Recall->",np.round(recall_score(y_train,y_pred),2))
print("F1-score->",np.round(f1_score(y_train,y_pred),2))

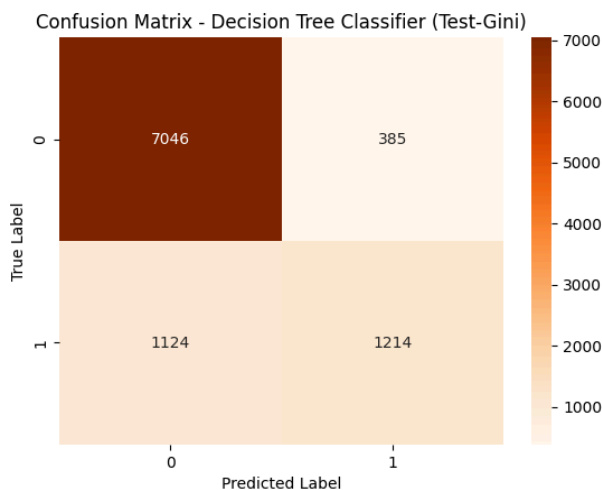
sns.heatmap(cm, annot=True, cmap='Oranges', fmt='g')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Decision Tree Classifier (Train-Gini)')
plt.show()
```



Training Portion:
Accuracy-> 0.84
Precision-> 0.76
Recall-> 0.51
F1-score-> 0.61

```
y_pred=dt.predict(X_test)
cm=confusion_matrix(y_test,y_pred)
print("Testing Portion")
print("Accuracy->",np.round(accuracy_score(y_test,y_pred),2))
print("Precision->",np.round(precision_score(y_test,y_pred),2))
print("Recall->",np.round(recall_score(y_test,y_pred),2))
print("F1-score->",np.round(f1_score(y_test,y_pred),2))

sns.heatmap(cm, annot=True, cmap='Oranges', fmt='g')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Decision Tree Classifier (Test-Gini)')
plt.show()
```



Testing Portion
Accuracy-> 0.85
Precision-> 0.76
Recall-> 0.52
F1-score-> 0.62

